

Appendix A:

Our goal is to assign the letters of the English language to the keys of the piano keyboard such that: (1) the pianist can use the same note transitions for entering text as when playing music, (2) dissonant and infrequent musical intervals are avoided and (3) frequent letter pairs (bigrams) are close together and can be played within one hand.

We formulate this problem in terms of placing items on a 1-D grid and use a greedy algorithm with additional constraints to create the letter-to-key mapping. To match the requirements we make use of the fact that there are more keys on the piano than letters in the alphabet and allow for a *one-to-many* mapping. This means that one letter can be mapped to multiple piano keys (the *redundancy* concept). The algorithm *constructs* the mapping from scratch, by assigning letters to keys one after another, following three steps:

1. **Map by frequency:** Map letters to keys in the order given by the frequency distributions of language and music.
2. **Check interval:** After each mapped letter, check if frequent bigrams containing this letter are mapped to note transitions common in music. If this is not the case, remap the letter to the next most frequent note and check again.
3. **Check distance:** After mapping all letters once, check the inter-key distance of frequent letter pairs. If the distance is too large, reduce it by mapping the more frequent letter to an additional key.

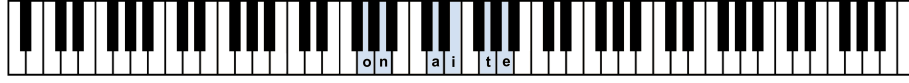
The following pseudocode describes the implementation of these principles.

Algorithm 1 FIND MAPPING

INPUT: 1- and 2-gram distributions of music and language
OUTPUT: a mapping of each letter to a set of notes

```
1: procedure FINDMAPPING(orderedLetters, orderedNotes)
2:   for all l in orderedLetters do
3:     mapping(l) := next(orderedNotes);                                ▷ Step (1)
4:     while !(checkInterval) do                                       ▷ Step (2)
5:       replace assignment with next most frequent note
6:     end while
7:   end for
8:   for all frequent bigrams do
9:     if !(checkDistance) then                                         ▷ Step (3)
10:      assign additional note
11:    end if
12:  end for
13: end procedure
```

Let us go through the algorithm step by step with help of examples. There are two main loops. The first one goes through all letters in the order of their frequency and assigns each to one piano key. Let us assume we are in the 7. iteration. The mapping so far would look like this:

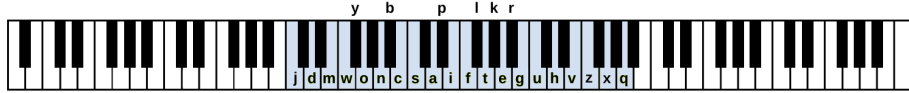


The next to-be-assigned letter, according to its frequency, is the letter ‘s’. In line 3 it is assigned to the next most frequent note of the corpus, which is C4:



The while loop in line 4 goes through all frequent letter pairs consisting of ‘s’ and the letters mapped so far: ‘es’, ‘as’, ‘st’, ‘is’, ‘se’, ‘so’, ‘si’, ‘ts’, ‘ss’, ‘ns’, ‘os’, ‘us’. For each of the pairs, the interval of the corresponding notes is considered and checked for its frequency in the music corpus. For example ‘es’ corresponds to the note transition from A4 to C4. They form the interval of an augmented sixth, frequently used in music. If one of the letter pairs corresponds to an interval infrequent in music, for example a dissonant interval, the algorithm will try to reassign the letter to the next most frequent note. This would be done in line 5. However, for the letter ‘s’ all intervals are harmonic and frequent. Thus C4 is a valid assignment for the letter ‘s’ and the algorithm can go on to the next iteration. In this way, at the end of the first for loop (line 7), all letters are assigned to *one* note.

The second loop (line 8) considers *all* frequent bigrams in the order of their frequency. The corresponding keys are checked for their distance. If the interval is larger than 8 semitones (a perfect fifth), the more frequent letter is mapped to an additional piano key, such that a smaller interval is formed (Step 3). The additional key is chosen such that it minimizes the interval to the other letter while maximizing the distance to the redundant key. Let us consider the following mapping, where every letter is mapped only once:



To clarify the function of step 3, we will go through two iterations of the second for loop. The most frequent bigram is ‘th’. It is considered first. The corresponding notes, G4 and D4, have a distance of 7 semitones. D is the dominant in the G major scale and the fifth, formed by those notes, is a very frequent interval in music. Thus *checkDistance* in line 9 returns true and we can continue with the next letter pair. In the eighth iteration, the algorithm considers the bigram ‘ed’. The corresponding notes, A4 and D3, have a distance of 20 semitones, exceeding the distance limit of 7 semitones. Thus we go on to line 10 and assign an additional key to the more frequent letter, the ‘e’. To maximize the benefit, it is mapped to C#3, which is far away from the other key assigned to ‘e’, while minimizing the distance to the ‘d’:



In this way a mapping from single notes to single letters is constructed.

However, to generate the *split*-mapping that is implemented in PianoText, we needed to apply the algorithm twice. The idea was to ensure that most of the

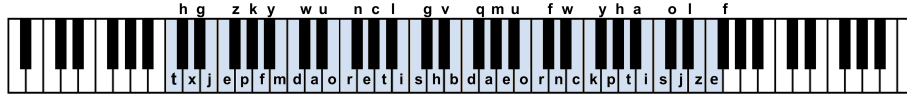


Figure 1: The mapping from letters to notes, as optimized by the algorithm.

keys are mapped at least twice, once on the right and once on the left side of the piano, such that it can be easily entered by each hand. Therefore the algorithm was first applied to the right part of the piano, starting from A3. Additionally only the most frequent bigrams were considered, occurring with a frequency greater 0.008 in the corpus. This ensures that the stronger right hand can reach all letters and most frequent bigrams. In the second run, the whole piano was considered as well as less frequent bigrams (frequency of 0.0008). In this way, nearly all letters were assigned an additional key on the left part of the piano. Only the letters in the range of A3 to E4, reachable by both hands, were not necessarily mapped twice. The outcome of this procedure can be seen in Figure 2. Note that the mapping does not utilize all the 88 keys of the piano. The ones on the far left and far right are used very infrequently in our music corpus and are thus excluded from the mapping. Very frequent letters, such as the ‘e’, are mapped three or four times, at least once on each side of the piano. More infrequent letters, such as ‘b’ or ‘v’ are mapped only once, but in the middle of the piano and thus reachable by each hand.

Figure ?? shows the mapping from n-grams to chords as used in Study 2. The n-grams were chosen in such a way that they optimize the performance benefit of the chord. The more frequent and longer an n-gram, the higher the gain for entering it with only one stroke. In addition we favored those n-grams that were particularly slow to enter with single key presses, such as ‘take’ or ‘yes’. We estimated the gain of a chord mapped to a certain n-gram with the following equation:

$$G(\text{chord}, n\text{-gram}) = (T_{n\text{-gram}} - T_{\text{chord}}) \cdot P(n\text{-gram}) \quad (1)$$

where $T_{n\text{-gram}}$ denotes the time to enter a n-gram by single key presses, T_{chord} the time to enter a chord. $P(n\text{-gram})$ denotes the frequency of the n-gram in a given corpus. The time it takes to press a chord was assumed to be constant. To estimate the time for entering a n-gram we looked at the average inter-key intervals from study 1. The frequency of n-grams was taken from <http://www.sketchengine.co.uk/>.

the	G	B	use	#F - D	ate	G - #D
that	E	G	over	D - B	because	E - C
th	G	#A	inter	G - C	yes	E - #G
and	#F	A	but	E - A	make	G - C - E
to	#D	G	not	F - D	ly	D - #A
in	B - D		ent	B - G	con	E - G - B
ment	F - A		would	#D - C	could	E - #C
with	#A - D		what	C - C	of	#D - B
ing	C - #D		by	D - D	take	C - E - G
this	C - E		will	F - F	ght	B - E - G
which	A - C		other	A - A	how	C - E - A
there	C - G		ter	B - B	about	#F - B
up	D - F		nes	G - E	when	E - A - C
where	F - C		one	C - #G	ted	B - E
pl	#C - E		from	B - E - #G	like	F - #A
him	D - G		ver	#A - D - G	her	#A - F
have	E - B		between	C - A	just	B - #F
able	D - A		these	G - B - D	good	E - G - C
for	G - D		tion	F - #A - D	many	A - D

Figure 2: The mapping from n-grams to chords as used in study 2. The letters after each n-gram describe the notes that form one chord. They can be played in any octave.